

Ch.5 Kubernetes 네트워킹

다음 날 아침이었습니다. 자리에 앉아 가방을 내려놓고 어제 띄워둔 Pod 네 대를 다시 떠올렸습니다. 자동 복구와 무중단 교체가 두 눈앞에서 돌아갔던 장면이 아직 또렷했습니다.

그때 옆자리 동료가 의자를 밀고 와 모니터를 들여다봤습니다.

동료 : "오픈 씨, 어제 띄운 거 어떻게 들어가요? 주소 좀 알려 주세요."

오픈이는 입을 떼려다 말았습니다. 정작 그 한마디가 막혔습니다.

'어, 어디로 들어가지...'

`kubectl get pods` 를 다시 두드렸습니다. Pod 네 대가 각자 다른 IP를 들고 줄지어 떠 있었습니다. 어제는 잘 돌아간다는 사실에만 들떠 있었는데, 막상 누군가에게 들어가는 문을 보여 주려니 손에 잡히는 주소가 한 개도 없었습니다.

게다가 Pod IP는 재시작 때마다 바뀝니다. 동료에게 IP 한 줄을 적어 주는 순간, Pod가 한 번이라도 죽었다 살아나면 그 주소는 사라진 번호가 됩니다.

'도커 때 Nginx 한 대를 앞에 세웠던 것처럼, 변하지 않는 문 하나가 있어야 하는데.'

오픈이는 다시 공식 문서를 펼쳤습니다.

5.1 Service - Pod의 고정 주소

5.1.1 Service가 필요한 이유

문서를 훑던 오픈이의 눈에 **Service**라는 항목이 들어왔습니다. Pod의 상태나 위치와 상관없이 변하지 않는 고정 진입점을 제공하는 자리, 그 일을 맡는 리소스가 Service였습니다.

가맹점이 많은 프랜차이즈를 떠올려 보면 이해가 쉽습니다. 매장 직원이 누구로 바뀌든 점장이 자리를 옮기든, 손님은 **브랜드 대표 번호** 하나로 언제나 주문을 넣을 수 있습니다. 직원의 휴대폰 번호가 아니라 매장에 불박이로 걸려 있는 번호라, 사람이 바뀌어도 그대로 남습니다.

Service가 클러스터 안에서 같은 자리를 맡습니다. Pod가 죽고 새로 뜨면서 IP가 바뀌어도, 그 앞에 걸린 대표 번호는 흔들리지 않습니다.

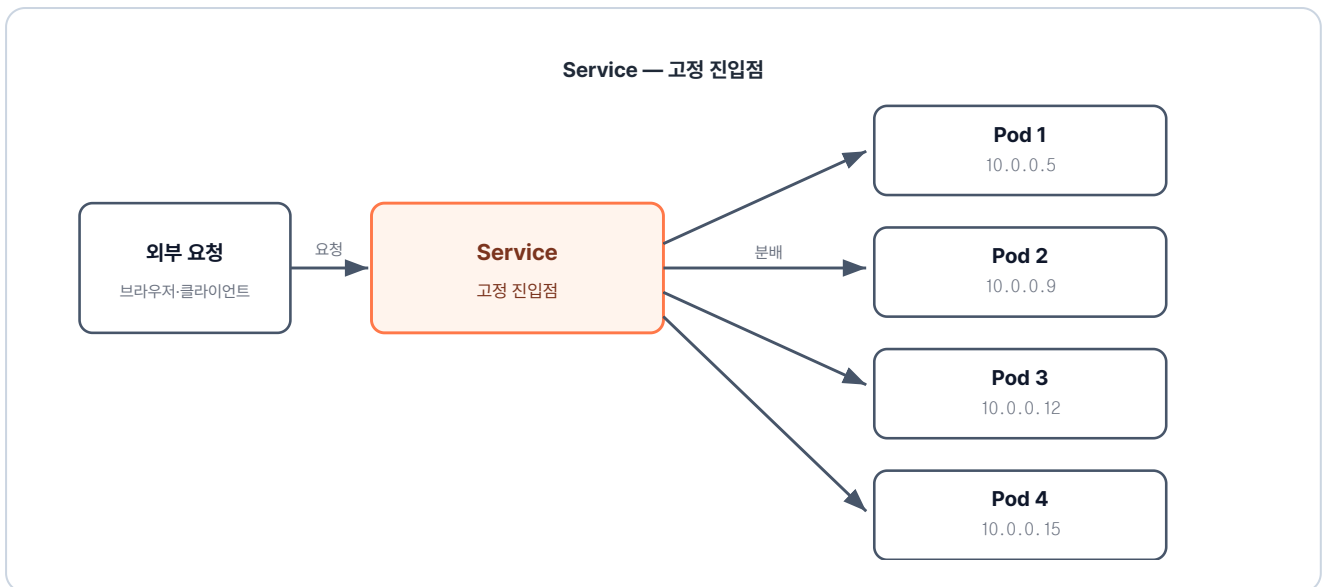


그림 5-1. Service는 Pod IP가 바뀌어도 변하지 않는 고정 주소를 제공

5.1.2 Service 생성

전체 실습 코드는 깃허브를 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex11>

오픈이는 직접 띄워 보기로 했습니다. 먼저 Pod를 세우고 그 앞에 대표 번호를 겁니다.

오픈이는 `ex11` 폴더를 열었습니다. 어제 사용했던 `deploy-ex02.yml` 이 그대로 남아 있습니다. 이 파일은 Pod 네 개를 띄우는 Deployment 설정입니다.

```
kubectl apply -f ex11/deploy-ex02.yml # Pod 4개 생성
```

오픈이는 다음으로 Service YAML을 살펴봤습니다.

ex11/service-ex01.yml

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort      # 노드 IP와 포트로 외부 접근이 가능한 타입
  selector:
    app: nginx        # 이 라벨을 가진 Pod들을 뒤에 연결
  ports:
    - port: 80        # 클러스터 내부에서 Service로 진입하는 포트
      targetPort: 80   # Service가 Pod로 요청을 전달하는 포트
      nodePort: 30080  # 외부에서 노드로 진입할 때 사용하는 포트 (기본 허용 범위 30000~3276

```

7)

핵심은 `selector` 한 줄입니다. Service는 Pod의 IP를 외워두지 않습니다. 대신 라벨로 식구를 가립니다. `app: nginx` 라벨이 붙은 Pod라면 IP가 무엇이든 뒤로 묶습니다. Pod가 죽고 새로 뜨면서 IP가 10.0.0.5에서 10.0.0.13으로 바뀌어도, 라벨만 같으면 Service는 그 Pod를 한 식구로 인식합니다.

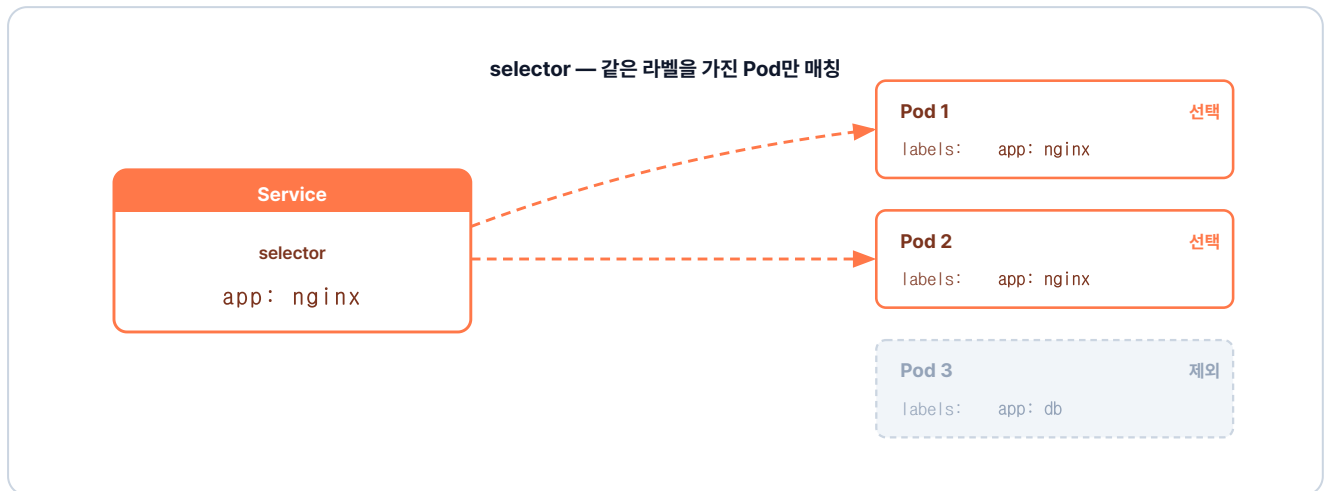


그림 5-2. selector가 지정한 라벨(app: nginx)을 가진 Pod만 매칭하고 다른 라벨은 제외

라벨이 일치하는 Pod 두 대만 Service 뒤로 묶이고, 다른 라벨이 붙은 Pod는 명단에서 빠집니다. 어제 Deployment를 만들 때 `app: nginx` 라벨을 매겼던 이유가 여기서 회수됩니다. 그 라벨이 오늘은 Service의 식구 명부 역할을 합니다.

5.1.3 Service 타입과 접근 범위

YAML을 다 작성하기 전에 오픈이는 `type: NodePort` 한 줄에서 멈췄습니다.

'NodePort 말고 다른 type도 있다는 거네. 어떤 차이가 있을까.'

문서를 더 들여다보니, Service는 누구에게 문을 여느냐에 따라 세 종류로 나뉘어 있었습니다. 집에 비유하자면 셋 다 문이긴 한데, 누구에게 열려 있는지가 다릅니다. 가족끼리만 쓰는 방문이 **ClusterIP**, 초대장을 받은 손님에게만 알려주는 옆문 비밀번호가 **NodePort**, 누구나 들어올 수 있는 정문이 **LoadBalancer** 입니다.

① ClusterIP

서비스 타입의 기본값입니다. 외부에서는 아예 접근이 불가능하며, 클러스터 내부의 Pod끼리 서로를 호출할 때만 사용합니다.

DB Pod 같은 경우가 여기에 해당합니다. 백엔드만 DB에 붙으면 되지, 외부 손님이 DB에 직접 들어올 일은 없습니다. 외부 노출이 필요 없는 구성에 ClusterIP를 씌우면, 안에서만 통하는 내선 번호가 생깁니다.

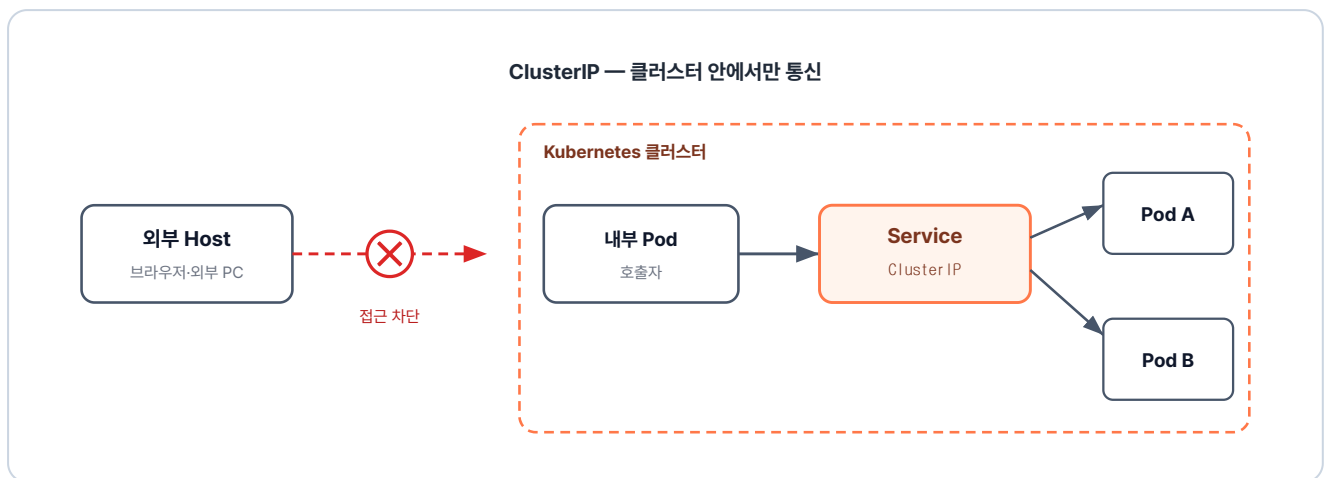


그림 5-3. ClusterIP는 외부 요청은 닿지 못하고 내부 Pod끼리만 통신

② NodePort

이번 실습에서 사용한 방식입니다. 노드(서버) 자체에 특정 포트를 열어 외부에서 들어올 수 있게 합니다.

YAML에 `nodePort: 30080` 이라고 적으면 노드의 30080번 포트가 열립니다. 그 포트로 들어온 요청이 클러스터 내부의 Service로 흘러갑니다. 옆문 비밀번호 30080을 아는 손님만 들어올 수 있는 구조입니다. 다만 서비스가 늘어날수록 비밀번호가 30080, 30081, 30082처럼 줄줄이 늘어나서, 누가 어느 문을 쓰는지 외우는 일이 만만치 않아집니다.

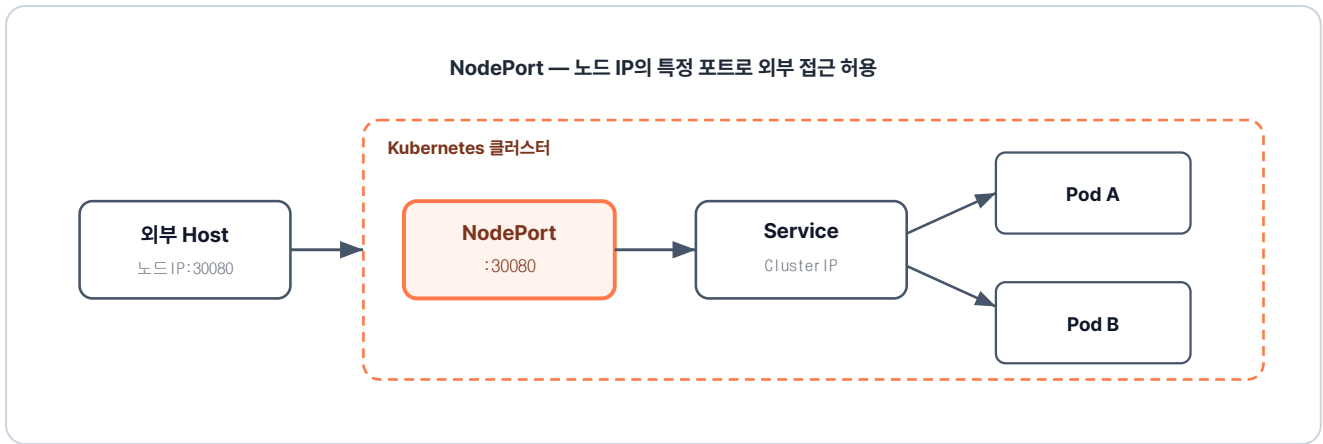


그림 5-4. NodePort는 노드의 특정 포트로 외부 접근을 허용

③ LoadBalancer

실제 운영 환경에서 가장 흔히 쓰는 방식입니다. AWS나 GCP 같은 클라우드 위에서 돌리는 클러스터라면, 쿠버네티스가 클라우드 사업자에게 공인 IP 하나를 발급받아 Service에 붙여 줍니다. 사용자는 30080 같은 포트 번호를 따로 외울 필요가 없고, 발급된 IP나 도메인 하나로 곧장 들어옵니다.

다만 사용자 눈에 보이지 않는 안쪽에서는 NodePort와 ClusterIP가 함께 만들어집니다. LoadBalancer 타입은 그 위에 공인 IP를 한 겹 더 얹은 형태로, 외부 트래픽은 공인 IP → 노드의 NodePort → kube-proxy → Pod 순으로 흘러갑니다.

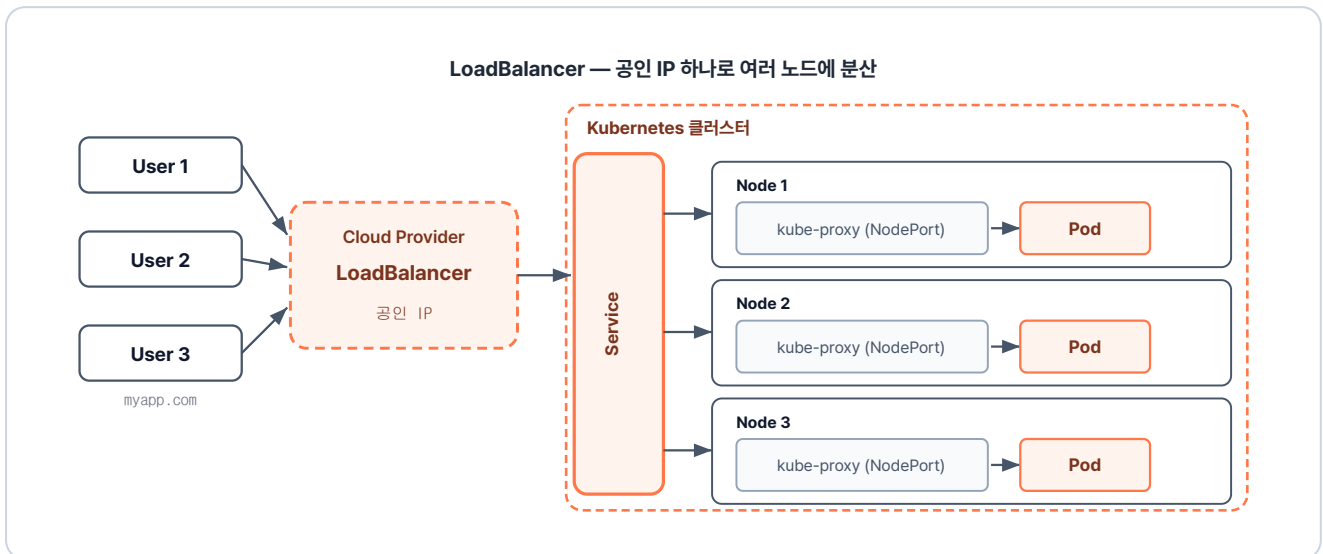


그림 5-5. LoadBalancer는 클라우드가 공인 IP를 발급해 여러 노드에 분산

세 타입의 차이를 한 표로 정리하면 다음과 같습니다.

타입	접근 범위	사용 사례
ClusterIP	클러스터 내부만	백엔드·DB 등 외부 노출이 불필요한 서비스
NodePort	노드IP:포트로 외부 접근 가능	테스트 및 개발 환경
LoadBalancer	공인 IP로 외부 접근 가능	실제 클라우드 운영 환경

타입은 정리됐지만 YAML에 적힌 `port`, `targetPort`, `nodePort` 라는 세 포트가 한 자리에 모여 있는 게 여전히 어색했습니다.

포트 흐름

'세 개가 다 80, 30080, 80인데 무슨 차이지.'

다시 들여다보니 셋은 서로 주인이 다른 포트였습니다.

- `nodePort` 는 노드(서버)의 포트입니다.
- `port` 는 Service 자체의 포트입니다.
- `targetPort` 는 실제 Pod의 포트입니다.

외부에서 Pod까지 닿는 라우팅 경로를 한 YAML에서 한꺼번에 선언하니, 자연스럽게 줄지어 적히게 됩니다.

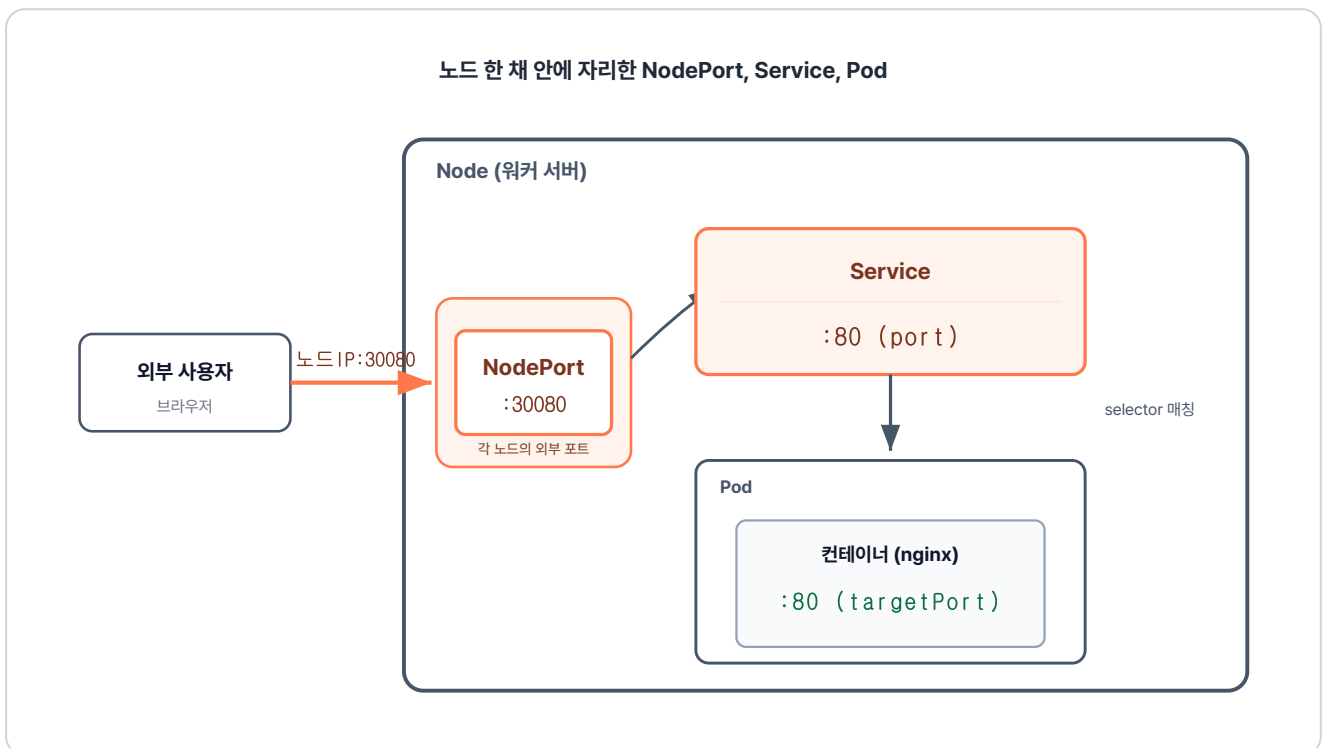


그림 5-6. 노드 안에 NodePort, Service, Pod가 자리하고 외부 사용자가 NodePort로 진입해 Service를 거쳐 Pod 안 컨테이너에 닿는 구조

포트 종류	소유 주체	역할	생략 시
<code>nodePort</code>	노드(서버)	외부에서 노드 IP로 접근할 때 열리는 포트	30000~32767 중 하나로 자동 할당. 명시 지정도 같은 범위
<code>port</code>	Service	클러스터 내부에서 Service를 부를 때 쓰는 포트	생략 불가 (필수)
<code>targetPort</code>	컨테이너	컨테이너 앱이 사용하는 포트와 일치해야 함	생략 시 <code>port</code> 와 같은 값

5.1.4 외부에서 Service 접속해 보기

Service가 정말 고정 진입점 역할을 하는지 직접 확인해야 어제 막혔던 자리가 풀립니다. 외부에서 들어가 보고, Pod가 죽고 IP가 바뀌어도 같은 주소로 닿는지 차례로 시험해 보겠습니다.

이제 작성한 YAML을 클러스터에 적용해 보겠습니다.

```
kubectl apply -f ex11/service-ex01.yml # Service YAML 적용
```



실행 결과

```
$ kubectl apply -f ex11/service-ex01.yml
service/nginx-service created
```

그림 5-7. Service 생성 결과

`nginx-service created` 라는 한 줄이 떴습니다. 이제 동료에게 보여줄 주소가 생긴 것 같았습니다. 오픈이는 곧장 브라우저를 열고 `localhost:30080` 을 입력했습니다.

화면이 멈춰 있었습니다. 빈 페이지가 한참 동안 그대로였습니다.

'분명 30080 포트를 열었는데 왜 안 들어가지.'

번뜩, 어제 미니큐브를 처음 띄울 때 봤던 메시지가 떠올랐습니다. 미니큐브는 컴퓨터 안에서 가상 환경을 한 겹 두르고 돌아갑니다. 노트북이 사는 동네와 미니큐브가 사는 동네가 서로 다른 네트워크라, 30080을 미니큐브 쪽에 열어 두어도 노트북 쪽에서는 그 포트가 보이지 않습니다. 도커에서 `-p`로 호스트와 컨테이너 사이에 길을 뚫었던 것처럼, 미니큐브와 노트북 사이에도 별도의 길이 필요합니다.

다행히 미니큐브는 이 둘을 잇는 통로를 만드는 명령어를 따로 마련해 두고 있습니다.

방법	명령어	설명
URL 생성	<code>minikube service <서비스이름> --url</code>	NodePort 또는 LoadBalancer 타입의 Service에 접근할 수 있는 URL 생성
터널 개방	<code>minikube tunnel</code>	LoadBalancer 타입 서비스에 외부 IP 부여

NodePort 타입에는 `minikube service --url`이 가장 간단합니다. 한 번 실행하면 노트북에서 들어갈 수 있는 URL을 발급해 줍니다.

```
minikube service nginx-service --url # Service 접근 URL 생성
```

실행 결과

```
$ minikube service nginx-service --url
http://127.0.0.1:10345
! windows 에서 Docker 드라이버를 사용하고 있기 때문에, 터미널을 열어야 실행할 수 있습니다.
```

그림 5-8. minikube service URL 생성 결과

URL이 한 줄 떴고 커서는 그대로 멈춰 있습니다. 이 명령어는 통로를 살려두기 위해 터미널을 점유한 채로 머물러 있습니다. 발급된 주소를 그대로 브라우저에 붙여 넣자, 익숙한 NGINX 환영 페이지가 모니터에 가득 찼습니다.



그림 5-9. 브라우저에서 nginx 접속 확인

이번엔 Pod 변동을 시험해 볼 차례였습니다. Pod가 죽고 IP가 바뀌어도 Service 뒤로 새 Pod가 자동 연결되는지 직접 보기로 했습니다. 오픈이는 통로를 닫고 Pod를 전부 지웠습니다. Deployment가 살아 있으니 곧 새 Pod가 자동으로 올라왔습니다.

```
kubectl delete pod --all          # 모든 Pod 삭제 (Deployment가 자동 재생성)
minikube service nginx-service --url # 다시 접속 URL 확인
```



그림 5-10. Pod 삭제 후 Service 접속 확인

새 URL로 접속하니 NGINX 페이지가 그대로 떴습니다. Pod IP가 전부 바뀌었어도 Service는 새 Pod에 이미 연결되어 있었습니다.

'와, 진짜네. Pod가 새로 만들어지면 IP가 바뀌었을 텐데. Service가 뒤에서 주소를 알아서 연결해주는구나.'

5.2 Ingress - 도메인 라우팅

5.2.1 Service의 한계

그때 옆자리 의자가 다시 굴러왔습니다. 동료가 화면을 살피며 "이 주소로 사람들이 쉽게 들어올 수 있겠어요?"라고 물었습니다.

오픈이는 발급된 `127.0.0.1:10345`를 다시 봤습니다. 터미널을 닫는 순간 사라지는 주소였습니다. 다시 띄울 때마다 포트도 바뀌었습니다.

'안에서는 Service로 묶었는데, 밖에서 들어올 입구는 아직 임시구나.'

Service를 하나 더 띄우면 입구도 또 하나 늘어납니다. 손님 입장에서 입구가 두 개, 세 개로 늘어난다는 뜻입니다. 그렇다고 임시 포트를 사람마다 알려 줄 수도 없습니다.

'도커 때 NGINX가 그 자리에 있었지.'

도커에서는 NGINX가 손님 앞에 서서 같은 도메인 아래 경로별로 뒤쪽 서버를 갈라 보내 줬습니다. 쿠버네티스에서도 그 자리가 필요합니다.

Service YAML을 처음부터 끝까지 다시 훑어봐도 경로별 라우팅 항목은 보이지 않았습니다. Service는 라벨로 Pod를 묶을 줄만 알지, URL 경로를 보고 길을 가르는 일은 하지 않습니다.

'Service 안에는 그런 기능이 없으니, 그 일을 맡는 다른 리소스가 따로 있는 건가.'

쿠버네티스 문서를 검색하다 곧 답이 나왔습니다. 그 자리를 맡는 전용 리소스의 이름이 **Ingress**였습니다.

5.2.2 Ingress의 역할

Service는 Pod 그룹마다 고정 진입점을 만들어 줍니다. 그러면 외부에서 들어올 때 Service의 수만큼 진입점을 알아야 합니다. 외부 도메인 하나로 들어와도 안에서 알아서 갈라져 가는 구조가 필요합니다.

Ingress가 이 분기를 자동으로 처리합니다. 외부 도메인 하나로 들어온 요청의 URL 경로를 보고 어느 Service로 보낼지 결정합니다. 우체국 물류 센터의 작업자가 우편의 바코드를 찍고 어느 우체국으로 보낼지 정해 주는 것과 같습니다.

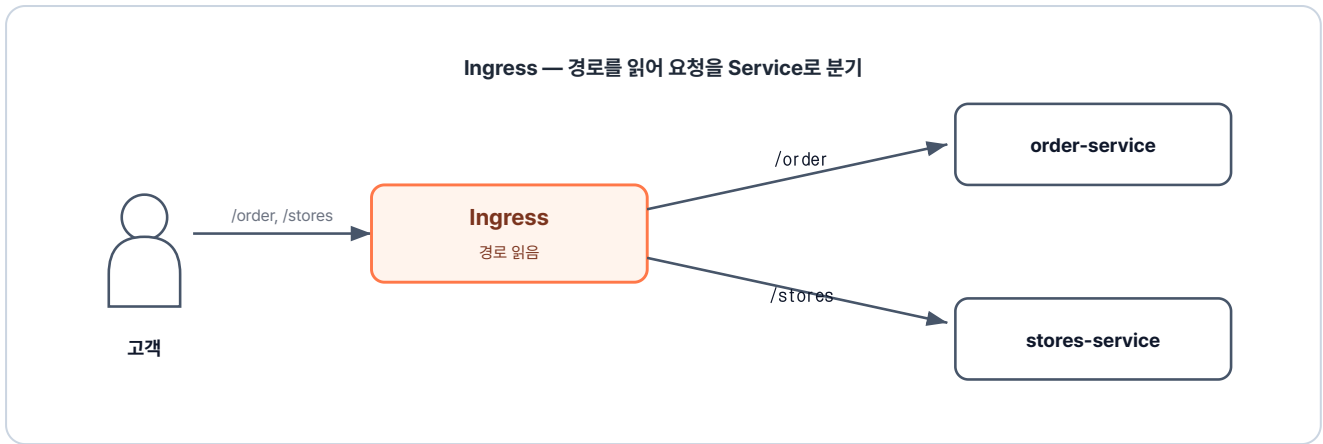


그림 5-11. Ingress가 도메인과 경로를 읽어 요청을 적절한 Service로 연결하는 구조

Ingress: 클러스터 외부의 HTTP/HTTPS 요청을 도메인과 URL 경로 기준으로 내부 Service에 연결하는 라우팅 규칙입니다. Service가 개별 Pod 그룹의 고정 주소를 제공한다면, Ingress는 여러 Service를 하나의 진입점으로 묶어 줍니다.

Service는 Pod 묶음 하나에 대표 번호를 달고, Ingress는 그 대표 번호 여러 개를 묶어 손님 쪽 진입을 한 곳으로 통일합니다. 한 단계 위에 또 한 명의 문지기를 두는 구조라, 외부 도메인 하나에 여러 Service를 매달 수 있습니다.

'외부 진입점 하나로 다 묶을 수 있다는 거네. 직접 띄워 봐야 감이 오겠다.'

5.2.3 Ingress 적용하기

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex12>

문서의 첫 줄을 읽던 오픈이의 눈이 한 곳에서 멈췄습니다.

"인그레스 컨트롤러가 있어야 인그레스를 충족할 수 있다. 인그레스 리소스만 생성한다면 효과가 없다."

'인그레스 리소스 말고 컨트롤러라는 게 또 따로 있어야 한다고?'

Ingress는 라우팅 규칙을 적은 **리소스(YAML)** 와 그 규칙을 읽고 외부 트래픽을 처리하는 **컨트롤러(Pod)** 두 가지로 분리됩니다.

리소스만 만들고 컨트롤러가 떠 있지 않으면 라우팅이 동작하지 않습니다. 두 가지가 함께 떠 있어야 작동합니다.

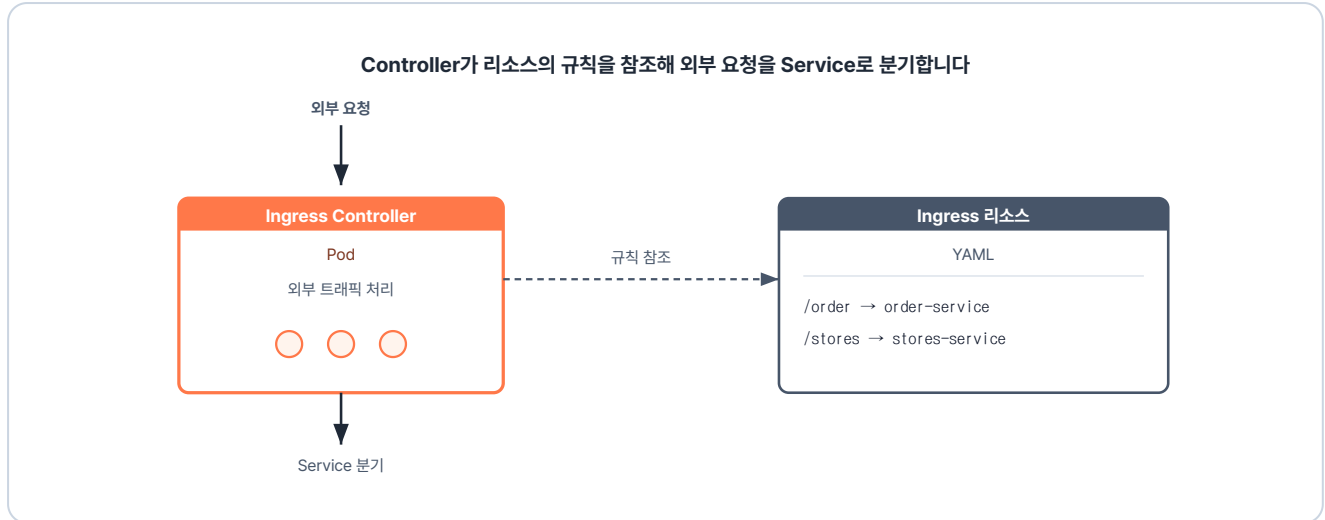


그림 5-12. Ingress 리소스(YAML)와 Ingress Controller(Pod) — 둘 다 있어야 라우팅이 작동합니다

'아, 규칙만 적어 둔다고 알아서 굴러가는 게 아니구나. 그 규칙을 읽고 실제로 실행해 주는 역할이 따로 필요한 거네.'

실습을 위해 미니큐브에 인그레스를 활성화해 보겠습니다. 미니큐브는 Ingress Controller를 애드온 형태로 한 번에 띄울 수 있는 명령어를 제공합니다.

```
minikube addons enable ingress # 인그레스 애드온 활성화
```

실행 결과

```

$ minikube addons enable ingress
* ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at:
https://github.com/kubernetes/minikube/blob/master/OWNERS
* After the addon is enabled, please run "minikube tunnel" and your ingress resources would
be available at "127.0.0.1"
  - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
  - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
  - Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
* Verifying ingress addon...
* The 'ingress' addon is enabled
  
```

그림 5-13. minikube에서 ingress 애드온 활성화 과정

마지막 줄에 `'ingress' addon is enabled`가 떴습니다. 컨트롤러 Pod가 실제로 떠 있는지 한 번 더 확인합니다.

```
kubectl get pods -n ingress-nginx # 컨트롤러 Pod 확인
```



실행 결과

```
$ kubectl get pods -n ingress-nginx
NAME READY STATUS RESTARTS AGE
ingress-nginx-admission-create-zg9hh 0/1 Completed 0 52m
ingress-nginx-admission-patch-bvxlw 0/1 Completed 1 52m
ingress-nginx-controller-9cc49f96f-qrpm7 1/1 Running 0 52m
```

그림 5-14. Ingress Controller Pod가 Running 상태임을 확인

`ingress-nginx-controller`가 `1/1 Running` 상태입니다. 물류 센터가 가동되어 우편을 받을 준비를 마쳤다는 뜻입니다.

① 두 서비스 준비

이제 들어온 요청을 받아 처리할 Service를 띄울 차례입니다. 주문 쪽과 매장 쪽을 각각 ClusterIP Service로 만들겠습니다. 두 Service는 같은 이미지(`hashicorp/http-echo`)를 쓰는데, `-text` 옵션으로 응답 문구만 달리 줍니다. 주문 쪽은 `"주문 접수 완료"`, 매장 쪽은 `"매장 선택"`이 돌아옵니다.

`ex12/` 안에는 주문과 매장 각각 Pod를 띄우는 **Deployment** 와 그 Pod를 묶는 **ClusterIP Service**가 한 쌍씩 들어 있습니다.

파일	종류	역할
<code>order-deploy.yml</code>	Deployment	주문 응답 Pod ("주문 접수 완료")
<code>order-service.yml</code>	Service (ClusterIP)	주문 Pod를 묶는 내부 창구 (port 5678)
<code>stores-deploy.yml</code>	Deployment	매장 응답 Pod ("매장 선택")
<code>stores-service.yml</code>	Service (ClusterIP)	매장 Pod를 묶는 내부 창구 (port 5678)

두 Service 모두 ClusterIP 타입이라 클러스터 바깥에서는 직접 접근할 수 없습니다. 부서 내선 같은 자리라, 손님이 직접 그 번호를 누르지는 못합니다. 손님 쪽 요청을 받아서 이 둘로 갈라 보낼 Ingress 규칙이 필요합니다.

㉔ 규칙 문서 작성

이제 컨트롤러에게 어떤 경로를 어디로 보낼지 정의한 지침서를 전달할 차례입니다.

ex12/ingress-ex01.yml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ex12-ingress
spec:
  ingressClassName: nginx          # 어느 Controller가 이 규칙을 집행할지 지정
  rules:
    - http:
        paths:
          - path: /order            # 주문 경로 → order-service
            pathType: Prefix
            backend:
              service:
                name: order-service
                port:
                  number: 5678
          - path: /stores           # 매장 경로 → stores-service
            pathType: Prefix
            backend:
              service:
                name: stores-service
                port:
                  number: 5678
```

`rules` 아래 두 개의 `path`가 있고, 각 경로가 서로 다른 Service를 가리킵니다. `/order`는 주문 Service로, `/stores`는 매장 Service로 돌리는 두 줄짜리 라우팅 규칙입니다. Ingress Controller가 이 규칙을 읽고 실제 트래픽을 분류해 보냅니다.

오픈이는 `ex12/` 폴더의 모든 파일을 적용해 두 Service와 Ingress 규칙을 한 번에 등록했습니다.

```
kubectl apply -f ex12/
kubectl get ingress          # 등록된 인그레스 확인
```



그림 5-15. Ingress 리소스 등록 확인

③ 브라우저로 접속

가상 세계와 원활히 통신하기 위해 별도 터미널에서 `minikube tunnel` 을 실행합니다.

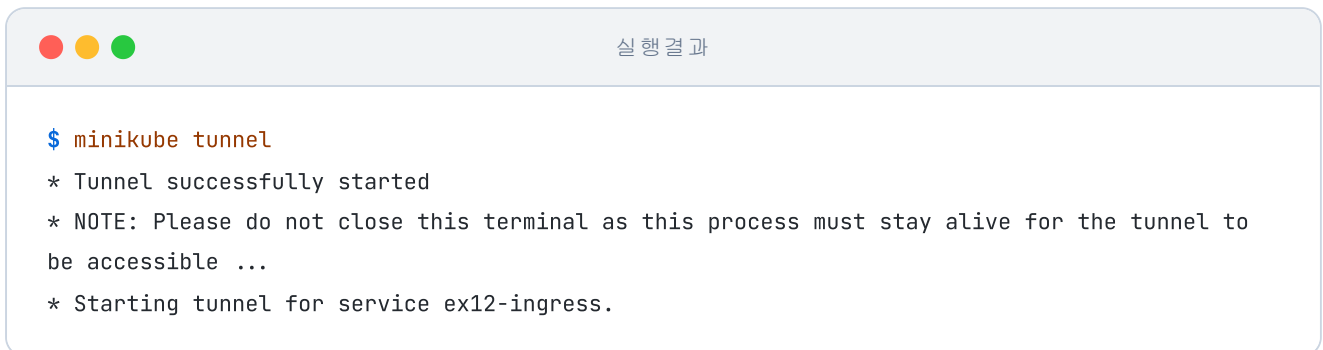
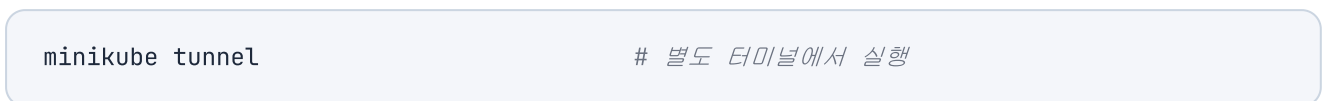


그림 5-16. minikube tunnel 실행 화면

이 명령은 Ingress의 80·443 포트를 호스트로 노출해 브라우저에서 `localhost` 로 바로 접속할 수 있게 해 줍니다. 이제 경로별로 직접 접속해 보겠습니다.

- `http://localhost/order` → "주문 접수 완료"
- `http://localhost/stores` → "매장 선택"

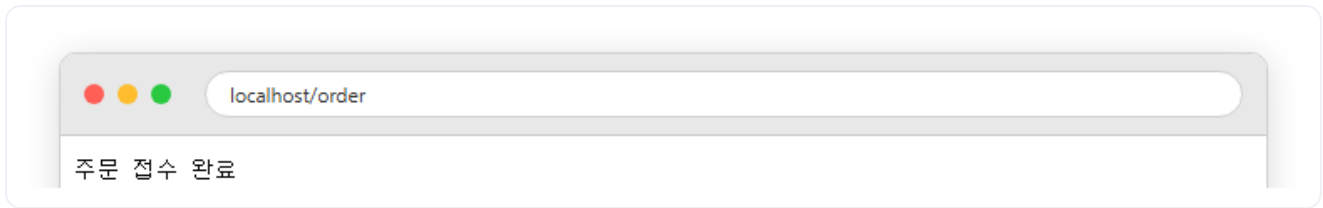


그림 5-17. /order 접속 결과 - "주문 접수 완료"

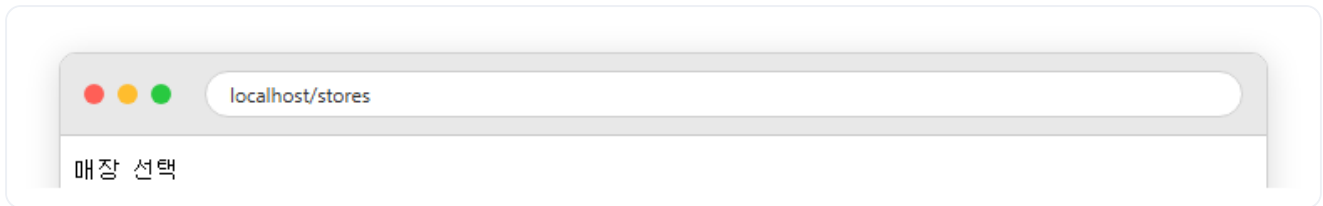


그림 5-18. /stores 접속 결과 - "매장 선택"

같은 도메인인데 뒤에 붙는 경로 한 글자에 따라 응답이 달라집니다. `localhost` 한 곳으로만 들어오는데도 누른 경로에 따라 주문 Service와 매장 Service로 정확히 갈라집니다. Ingress가 URL 경로를 읽고 알맞은 Service로 트래픽을 분기해 준 결과입니다.

동료가 등 뒤로 와서 화면을 들여다봤습니다.

동료 : "이번엔 주소 하나만 알려 주면 되는 거예요?"

오픈이는 `localhost/order` 와 `localhost/stores` 를 차례로 적어 줬습니다. 이번에는 포트 번호도 없고, 따로 외울 IP도 없었습니다.

5.2.4 Service와 Ingress가 보는 정보가 다른 이유

오픈이는 두 리소스가 같은 트래픽을 다루는데 보는 정보가 다르다는 점이 궁금했습니다. 우편 한 통이 떠올랐습니다.

우편 위 **받는 곳 주소**는 사람이 눈으로 읽는 위치 정보입니다. **송장**은 우체국이 시스템에 등록한 뒤 출력해 붙이는 바코드 라벨입니다. 분류 라인·도착 우체국 코드 같은 시스템적 분류 정보가 구조화되어 있어 스캐너로만 읽힙니다. 두 정보는 같은 우편 위에 있지만 누가 읽는지와 들이는 손이 다릅니다.

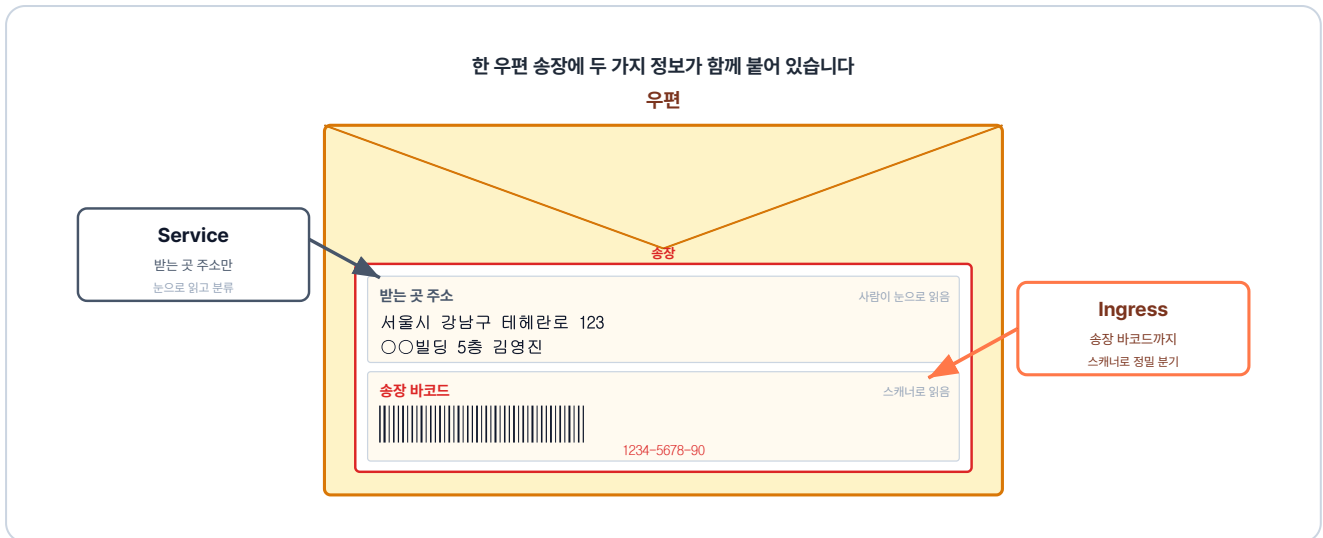


그림 5-19. 우편 위 송장에 받는 곳 주소와 바코드가 함께 있고, Service는 받는 곳 주소만, Ingress는 바코드까지 읽습니다

네트워크 패킷도 이 두 정보를 닮았습니다. **Service**는 받는 곳 주소에 해당하는 IP·포트만 보고 일합니다. 모든 노드 커널의 iptables 규칙으로 분류·전달합니다. **Ingress**는 한 단계 더 들어가, 송장 바코드에 해당하는 도메인·URL 경로까지 읽고 어느 Service로 보낼지 결정합니다. 별도 Pod 안 HTTP 서버가 이 일을 합니다.



그림 5-20. 실제 패킷에 TCP/IP 헤더와 HTTP 페이로드가 함께 들어 있고, Service는 헤더만, Ingress는 페이로드까지 읽습니다

항목	Service	Ingress
보는 정보	받는 곳 주소 (IP·포트)	송장 바코드 (URL·Host)
일하는 위치	모든 노드 커널 iptables	별도 Pod 안 HTTP 서버

5.3 브라우저에서 Pod까지의 경로

오후 늦게 욕상에 잠깐 올라갔습니다. 바람을 맞으며 머릿속 그림을 다시 한 번 정리하고 싶었기 때문입니다. NodePort, kube-proxy, Ingress Controller, ClusterIP. 이름은 다 들어봤는데 한 줄로 늘어놓고 보면 누가 어디에 서서 무얼 하는지가 흐릿합니다.

자리로 돌아와 흰 종이를 한 장 꺼냈습니다. 통합 사이트의 브라우저에서 Pod까지 한 번 죽 그어 봤습니다. 한 자리에 너무 많은 컴포넌트가 모이지 않게, 일을 세 단계로 잘랐습니다.

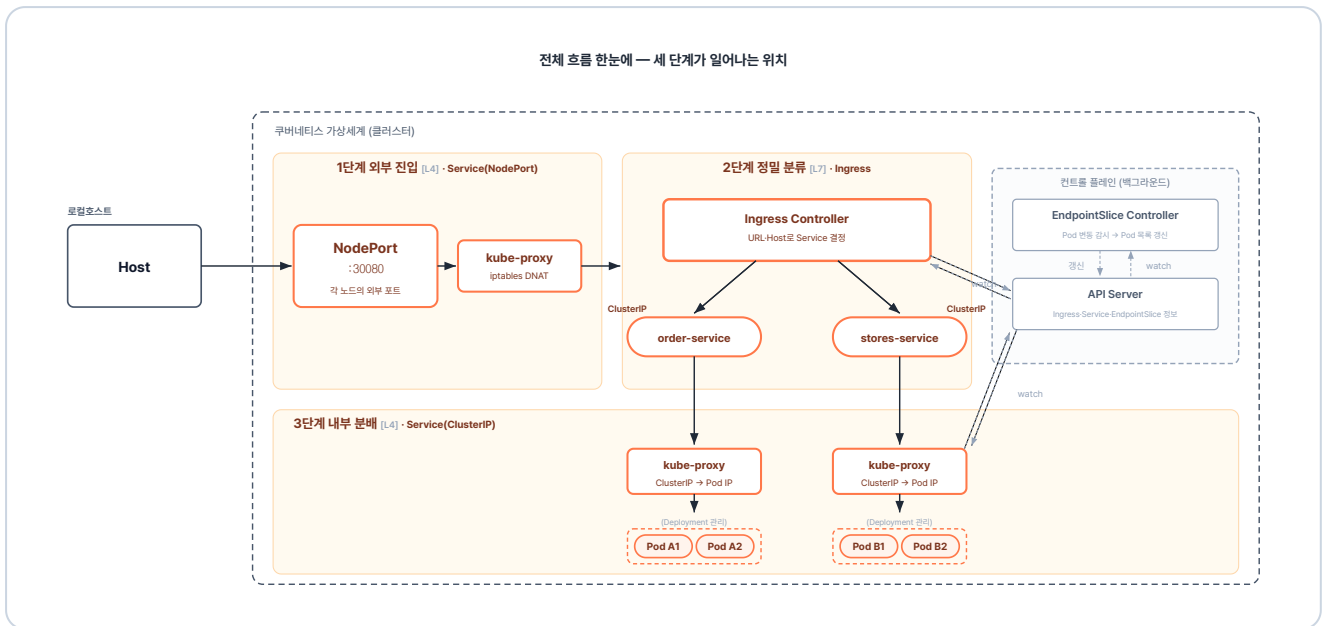


그림 5-21. 요청이 Pod까지 닿는 전체 흐름 - 세 단계가 일어나는 위치

kube-proxy: 모든 노드(서버)에 항상 떠 있는 프로그램입니다. 들어온 요청의 받는 주소가 살아있는 Pod의 주소로 바뀌도록 iptables 규칙을 미리 깔아 두고 갱신합니다.

이 흐름을 우체국에 우편을 보내는 예시를 통해 단계별로 살펴보겠습니다.

5.3.1 1단계 진입 - 우체국 입구에서 송장을 붙인다

발신자가 우편을 들고 동네 우체국 입구로 들어갑니다. 우편 봉투에는 동네 우체국으로 가는 외부 주소가 적혀 있습니다. 안쪽 **창구 직원**이 그 외부 주소를 가리고 그 위에 새 **송장**을 붙입니다. 새 송장 바코드에는 우편이 **다음으로 갈 물류 센터의 라우팅 코드**가 들어 있습니다. 우편에 적힌 받는 사람·주소는 그대로 따라가며 최종 목적지를 알려 줍니다.

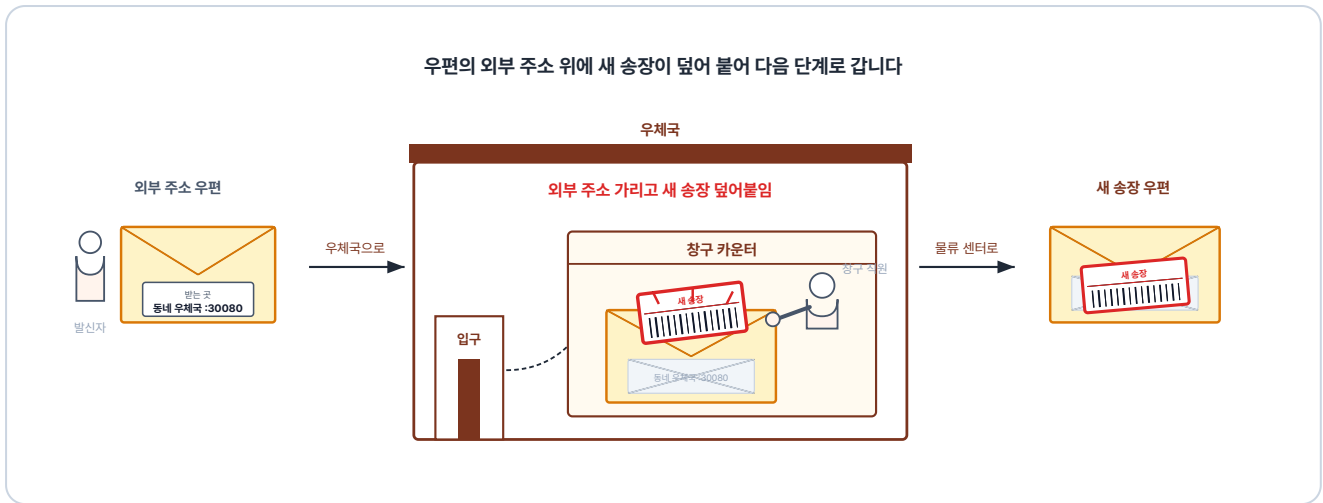


그림 5-22. 우체국 창구에서 우편의 외부 주소 위에 새 송장(바코드)이 덮여 붙어 다음 단계로 넘어갑니다

비유	IT 용어	한 줄 설명
발신자	외부 호스트	브라우저로 요청을 보내는 사용자
우체국 입구	NodePort	각 노드의 공개 포트
창구 직원	kube-proxy (1차)	iptables 규칙을 미리 깔아 두고, 그 규칙으로 받는 곳을 바꿔 다음 단계로 전달
송장	NAT 변환	패킷의 받는 곳을 Ingress Controller의 IP로 변환

브라우저가 `http://localhost:30080/order` 를 누르면 요청은 노드의 NodePort(`:30080`)로 들어옵니다. 같은 노드의 kube-proxy가 미리 깔아 둔 iptables 규칙을 따라 패킷의 목적지 IP가 Ingress Controller Pod의 IP로 변환됩니다.

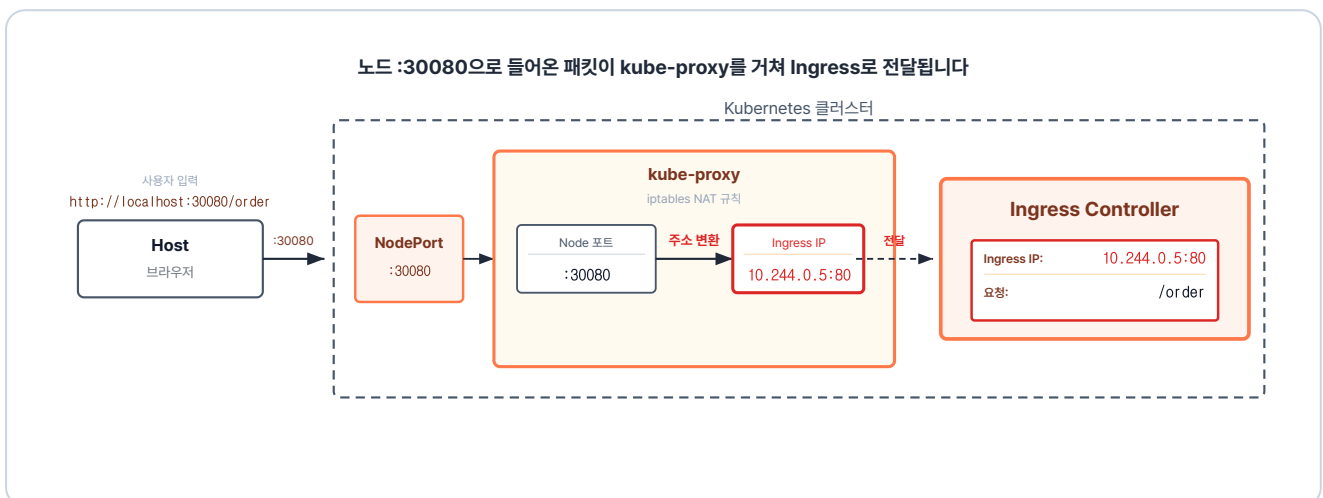


그림 5-23. NodePort와 kube-proxy(1차)가 외부 요청을 Ingress Controller로 전달

5.3.2 2단계 분류 - 물류 센터가 바코드를 보고 어디로 보낼지 정한다

송장이 붙은 우편이 **물류 센터**에 도착합니다. 작업자가 송장의 **바코드**를 스캐너로 찍으면 시스템이 옆에 비치된 **분류 규칙**에 따라 어느 우체국으로 갈지 곧장 알려 줍니다.

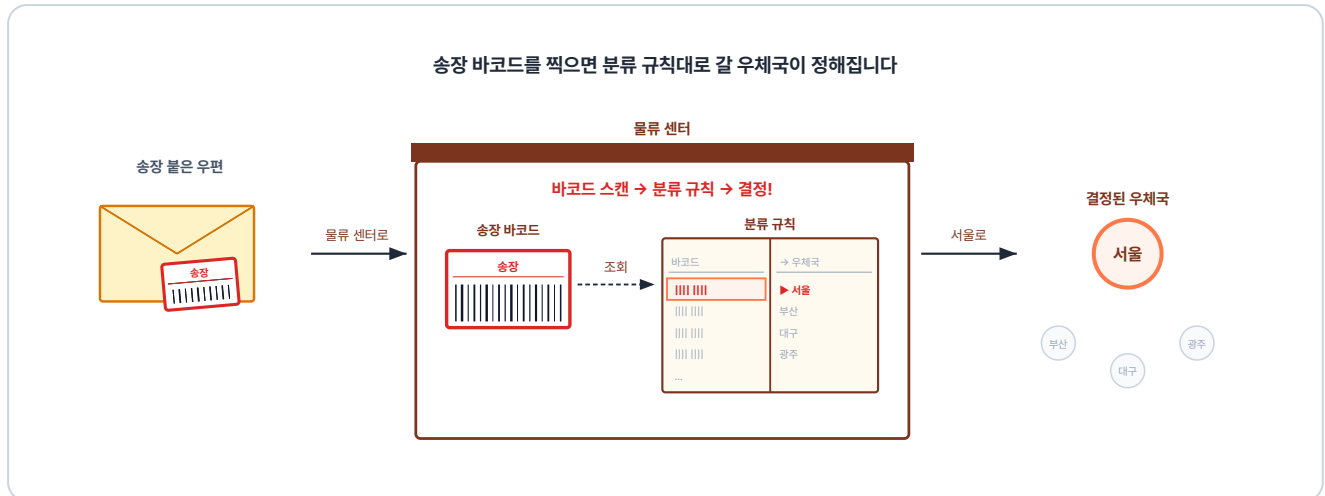


그림 5-24. 물류 센터가 송장의 바코드를 찍고 어느 우체국으로 보낼지 정합니다

비유	IT 용어	한 줄 설명
물류 센터	Ingress Controller	URL·Host로 보낼 Service를 결정
바코드	URL·Host 헤더	패킷의 L7 식별 정보
분류 규칙	Ingress 규칙	Controller가 받아 둔 라우팅 규칙
서울 우체국·부산 우체국	Service (ClusterIP)	셀렉터로 묶인 Pod 그룹의 가상 진입 주소

Ingress Controller는 요청의 URL 경로와 Host 헤더를 등록된 Ingress 규칙과 대조해 대상 Service를 결정하고, 해당 Service의 ClusterIP를 다음 목적지로 잡습니다.

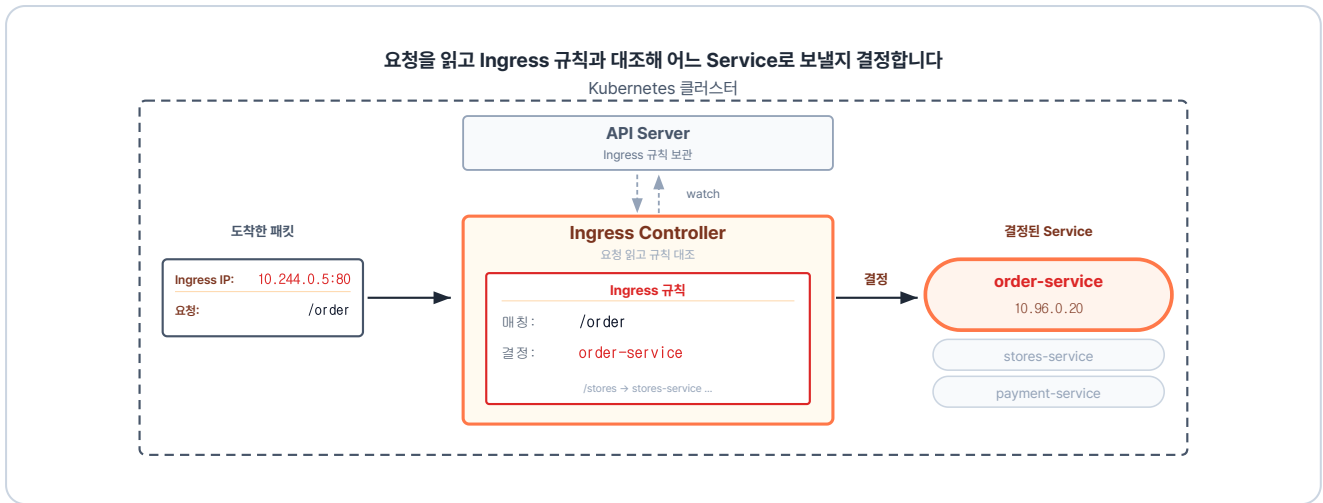


그림 5-25. Ingress Controller가 URL을 읽어 Service를 선택, 미리 받아 둔 분류 규칙에서 ClusterIP를 꺼냄

5.3.3 3단계 배달 - 집배원이 우편함에 넣는다

서울 우체국으로 도착한 우편을 그 동네 **집배원**이 받습니다. 집배원은 오늘 **배송 목록**을 들고 있습니다. 우편의 행선지를 그 목록과 맞춰 어느 **우편함**에 넣을지 정한 뒤 그 우편함에 우편을 넣습니다. 우편함이 우편의 종착지입니다.

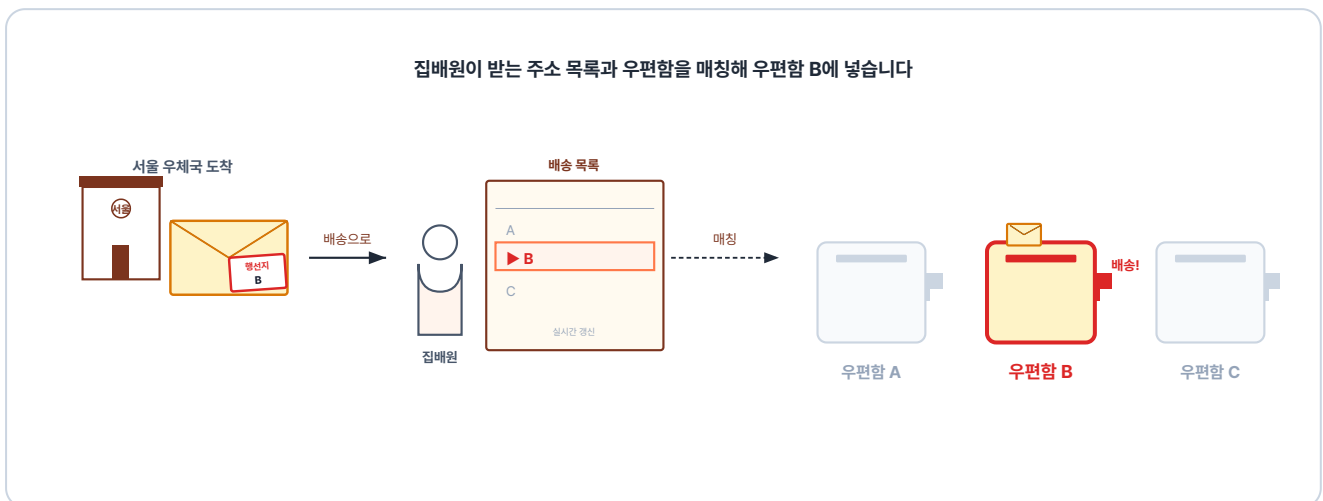


그림 5-26. 서울 우체국에서 집배원이 배송 목록을 보고 우편함 B에 우편을 넣습니다

비유	IT 용어	한 줄 설명
서울 우체국	Service (ClusterIP)	도착한 ClusterIP의 진입점
집배원	kube-proxy (2차)	iptables DNAT으로 ClusterIP를 살아있는 Pod IP로 변환
배송 목록	EndpointSlice	살아있는 Pod IP를 모은 명단
우편함	Pod	실제 비즈니스 로직을 처리하는 앱

ClusterIP를 향한 요청은 노드의 kube-proxy(2차)가 가로채, 미리 받아 둔 **EndpointSlice**의 살아있는 Pod IP 중 하나로 목적지를 변환(DNAT)합니다.

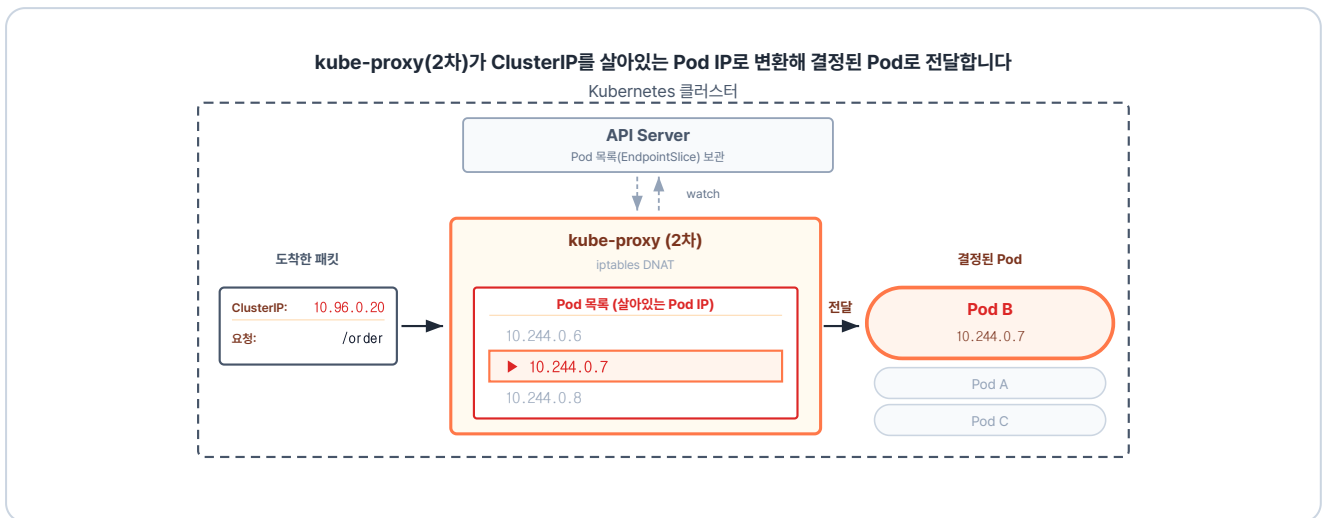


그림 5-27. kube-proxy(2차)가 미리 받아 둔 Pod 목록에서 살아있는 Pod IP를 골라 변환해 Pod로 전달

'우편이 입구에 들어와 송장이 붙고, 물류 센터에서 바코드를 보고 우체국이 정해지고, 그 동네 집배원이 우편함에 넣는다. 각자 한 가지 일만 하는데 이게 합쳐지니 전체가 굴러가네.'

이것만은 기억하자

- **Service는 Pod의 변하지 않는 고정 진입점입니다.** Pod는 소모품이라 IP가 수시로 바뀌지만, Service는 변하지 않는 주소를 제공하며 트래픽을 분산합니다.
- **Service 뒤에는 세 컴포넌트가 협력합니다.** 실시간 Pod 목록을 관리하는 **EndpointSlice Controller**, 명단을 노드에 전파하는 **API Server**, 실제 패킷을 Pod로 전달하는 **kube-proxy**가 함께 트래픽을 처리합니다.

- **Ingress는 외부 요청을 URL 경로로 분기합니다.** 숫자(IP·Port)만 보는 Service와 달리, Ingress는 도메인과 URL 경로를 읽고 적절한 Service로 연결합니다. 라우팅 규칙을 정의하는 **리소스(YAML)** 와 실제 요청을 처리하는 **컨트롤러(Pod)** 가 함께 동작합니다.

외부에서 들어오는 길은 이제 끊김 없이 깔립니다. 그런데 통합 사이트는 길만 있다고 굴러가지 않습니다. DB 비밀번호 같은 설정값과, Pod가 다시 떠도 사라지지 않는 데이터가 함께 있어야 합니다.

다음 챕터에서는 설정값(**ConfigMap**)과 보안 비밀(**Secret**), 데이터의 영속성(**Volume**)을 다뤄 보겠습니다.